

SPECYFIKACJA PROJEKTOWA: BIZNESOWA I TECHNICZNA

System Wspomagania Treningu Siłowego (FitSync / GymApp)

Wersja dokumentu: 1.0

Status: Projekt Architektury i Specyfikacji Wymagań

Docelowa platforma: Android (z wykorzystaniem frameworka React Native + Expo)

1. SPECYFIKACJA BIZNESOWA

1.1. Wizja i Cel Projektu

Głównym celem projektu jest wytworzenie lekkiej, intuicyjnej i w pełni niezależnej aplikacji mobilnej na platformę Android, przeznaczonej do planowania, realizowania i analizowania treningów siłowych. Aplikacja ma pełnić rolę cyfrowego dziennika treningowego zintegrowanego z ekosystemem biometrycznym Garmin, co pozwoli na śledzenie obciążenia treningowego w oparciu o obiektywne wskaźniki fizjologiczne.

1.2. Grupa Docelowa i Przypadki Użycia

- Osoby początkujące:** Korzystanie z gotowych szablonów treningowych, nauka poprawnej techniki dzięki opisom i materiałom graficznym, automatyczne prowadzenie przez czas trwania serii i przerw.
- Zaawansowani sportowcy siłowi:** Precyzyjne rejestrowanie tonażu, wagi, powtórzeń, subiektywnego zmęczenia (RPE), analiza progresji liniowej oraz śledzenie odczytów tętna/spoczynkowego z zegarka Garmin.

1.3. Zakres Funkcjonalny (Functional Requirements)

Moduł	Wymaganie biznesowe / Funkcjonalność	Priorytet (MoSCoW)
Słowniki i Ćwiczenia	Katalog ćwiczeń z opisami, instrukcją krok po kroku, grafiką/zdjęciem. Filtrowanie wg partie mięśniowe, sprzęt i poziom trudności.	MUST HAVE
Kreator Treningów	Tworzenie i edycja własnych planów: ustalanie serii, powtórzeń, ciężaru (kg), czasu trwania oraz przerw.	MUST HAVE
Szablony	Wbudowana baza domyślnych planów (FBW, Push-Pull-Legs, Trening Domowy).	MUST HAVE
Tryb Treningu na Żywo	Interfejs rejestracji serii w czasie rzeczywistym, ogólny stoper treningu, automatyczny timer przerwy z sygnałem.	MUST HAVE
Harmonogram i Kalendarz	Planowanie treningów w konkretne dni kalendarza, widok historii i planowanych sesji.	MUST HAVE
Powiadomienia i Mail	Przypomnienia lokalne (push) o treningu oraz możliwość wysyłania podsumowań/przypomnień na maila.	MUST HAVE

Moduł	Wymaganie biznesowe / Funkcjonalność	Priorytet (MoSCoW)
Historia i Korekta	Dziennik ukończonych treningów z możliwością dodawania uwag i notatek powykonywanych.	MUST HAVE
Analityka i Postępy	Wykresy objętości (tonażu), progresji ciężaru w czasie, szacowanie 1RM (One Rep Max) oraz historia wagi i wymiarów ciała.	SHOULD HAVE
Garmin Sync (Dwukierunkowa)	1. Wysyłanie planu/harmonogramu do Kalendarza Garmin. 2. Pobieranie danych biometrycznych (tętno, HRV, sen, stres, spalone kalorie, ciśnienie/odczyty zdrowotne).	MUST HAVE
Auto-progresja & RPE	Ocena RPE (1-10) po serii, kalkulator 1RM, sugestie korekty ciężaru na kolejny tydzień.	COULD HAVE

2. SPECYFIKACJA TECHNICZNA I ARCHITEKTURA

2.1. Architektura Systemu i Stos Technologiczny

- Warstwa Mobilna (Frontend):** React Native z frameworkiem Expo (Expo SDK 51+). Zapewnia natywny dostęp do funkcji systemu Android oraz ułatwiony proces budowania pliku `.apk`.

- **Lokalna Baza Danych:** SQLite (`expo-sqlite`) lub WatermelonDB / Realm. Gwarantuje pełne działanie aplikacji w trybie offline.
- **Stan Aplikacji:** Zustand lub Redux Toolkit.
- **Powiadomienia i Timery:** `expo-notifications` (lokalne alarmy i timery działające w tle).
- **Usługa E-mail (bez własnego backendu):** Integracja z REST API dostawcy usługi e-mail (np. Resend API lub EmailJS) uruchamiana z poziomu aplikacji.
- **Synchronizacja z Garmin:** OAuth 1.0a / OAuth 2.0 połączenie z Garmin Connect API / Garmin Health API.

2.2. Model Bazy Danych (Lokalna Baza SQLite)

Poniżej przedstawiono encje i relacje w lokalnej bazie danych:

```
[ExerciseCategory] 1 --- * [Exercise]
[Equipment]        1 --- * [Exercise]
[Exercise]          1 --- * [WorkoutPlanExercise]
[WorkoutPlan]       1 --- * [WorkoutPlanExercise]
[WorkoutPlan]       1 --- * [ScheduledWorkout]
[ScheduledWorkout] 1 --- 1 [WorkoutSession]
[WorkoutSession]    1 --- * [LoggedSet]
[WorkoutSession]    1 --- 1 [GarminBiometrics]
```

Struktura Schematu Bazy Danych (DDL)

```
-- 1. Kategorie i Słowniki
CREATE TABLE categories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  description TEXT
);

CREATE TABLE equipment (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    name TEXT NOT NULL
);
```

```
CREATE TABLE exercises (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category_id INTEGER REFERENCES categories(id),
    equipment_id INTEGER REFERENCES equipment(id),
    name TEXT NOT NULL,
    instructions TEXT,
    image_url TEXT,
    difficulty_level TEXT CHECK(difficulty_level IN
('BEGINNER', 'INTERMEDIATE', 'ADVANCED'))
);
```

-- 2. Plany i Szablony

```
CREATE TABLE workout_plans (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    is_template BOOLEAN DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE plan_exercises (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    plan_id INTEGER REFERENCES workout_plans(id) ON
DELETE CASCADE,
    exercise_id INTEGER REFERENCES exercises(id),
    order_index INTEGER NOT NULL,
    target_sets INTEGER NOT NULL,
```

```
    target_reps INTEGER,  
    target_weight REAL,  
    target_duration_seconds INTEGER,  
    rest_duration_seconds INTEGER DEFAULT 90  
);
```

-- 3. Harmonogram i Powiadomienia

```
CREATE TABLE scheduled_workouts (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    plan_id INTEGER REFERENCES workout_plans(id),  
    scheduled_date DATE NOT NULL,  
    is_completed BOOLEAN DEFAULT 0,  
    garmin_synced BOOLEAN DEFAULT 0  
);
```

-- 4. Wykonane Treningi i Logi

```
CREATE TABLE workout_sessions (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    scheduled_id INTEGER REFERENCES  
scheduled_workouts(id),  
    start_time DATETIME NOT NULL,  
    end_time DATETIME,  
    total_duration_seconds INTEGER,  
    user_notes TEXT,  
    rpe_rating INTEGER  
);
```

```
CREATE TABLE logged_sets (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    session_id INTEGER REFERENCES
```

```
workout_sessions(id) ON DELETE CASCADE,  
    exercise_id INTEGER REFERENCES exercises(id),  
    set_number INTEGER NOT NULL,  
    reps_completed INTEGER,  
    weight_kg REAL,  
    duration_seconds INTEGER,  
    completed_at DATETIME  
);
```

-- 5. Synchronizowane Dane Biometryczne Garmin

```
CREATE TABLE garmin_biometrics (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    session_id INTEGER UNIQUE REFERENCES  
workout_sessions(id),  
    summary_date DATE NOT NULL,  
    avg_heart_rate INTEGER,  
    max_heart_rate INTEGER,  
    resting_heart_rate INTEGER,  
    blood_pressure_systolic INTEGER,  
    blood_pressure_diastolic INTEGER,  
    calories_burned INTEGER,  
    stress_level INTEGER,  
    sleep_duration_minutes INTEGER,  
    hrv_status TEXT,  
    raw_garmin_json TEXT  
);
```

3. INTEGRACJA Z GARMIN CONNECT / HEALTH API

Integracja zakłada pełną dwukierunkową synchronizację danych pomiędzy aplikacją mobilną a ekosystemem Garmin.

3.1. Przepływ Synchronizacji Planów (Aplikacja -> Garmin)

1. Użytkownik tworzy lub przypisuje plan treningowy w harmonogramie aplikacji na dany dzień.
2. Aplikacja generuje plik struktury treningowej zgodny ze specyfikacją Garmin Training API (format JSON lub plik `.FIT`).
3. Zapytanie `POST` wysyłane do Garmin Connect API tworzy cel treningowy w kalendarzu Garmin użytkownika.
4. Podczas kolejnej synchronizacji zegarka z aplikacją Garmin Connect, trening pojawia się bezpośrednio na zegarku.

3.2. Przepływ Pobierania Danych Biometrycznych (Garmin -> Aplikacja)

Aby uzyskać dane o tętnie, ciśnieniu krwi, kaloriach czy poziomie stresu, aplikacja wykorzystuje dwustopniowy mechanizm:

- **Uwierzytelnienie:** Dedykowane okno OAuth w aplikacji mobilnej pozwala użytkownikowi zalogować się do konta Garmin i nadać uprawnienia do odczytu danych biometrycznych i aktywności (`Garmin Health API`).
- **Pobieranie Danych (PULL / Webhook Subscriptions):**
 - **Tętno i Kalorie aktywności:** Pobieranie szczegółowych danych z sesji aktywności po zakończeniu treningu (zapytanie o Activity Summary & Epoch Data dla konkretnego przedziału czasowego treningu).
 - **Dane Dobowe (Resting HR, Stres, Sen, Ciśnienie):** Pobieranie Dziennych Podsumowań (Daily Health Summary) z dnia treningowego. Za pomocą integracji z urządzeniami Garmin Index BPM pobierane są również pomiary ciśnienia tętniczego (skurczowe / rozkurczowe).
- **Mapowanie w Bazie:** Zwrócone wartości zapisywane są w tabeli `garmin\_biometrics` i wiązane relacją zID sesji treningowej.

Parametr Garmin	Metryka w aplikacji	Zastosowanie w analizie
Heart Rate (Avg/Max)	Średnie i maksymalne tętno sesji	Kalkulacja intensywności treningu siłowego.
Resting HR & HRV	Tętno spoczynkowe i zmienność rytmu serca	Ocena gotowości do treningu i regeneracji układu nerwowego.
Active Calories	Spalone kalorie podczas sesji	Dzienny bilans energetyczny.
Blood Pressure	Ciśnienie skurczowe / rozkurczowe	Śledzenie wpływu ciężkich treningów na układ krążenia.
Stress & Sleep Data	Poziom stresu i jakość snu z minionej nocy	Korelacja wydajności na treningu z regeneracją.

4. URUCHOMIENIE I DEPLOJAMENT NA TELEFONIE ANDROID4.1. Środowisko Deweloperskie i Testy Na Żywo

- **Narzędzie: Expo CLI (`npx expo start`).**
- **Instalacja na telefonie:** Aplikacja **Expo Go** (ze sklepu Google Play).
- **Sposób uruchomienia:** Komputer i telefon podłączone do tej samej sieci Wi-Fi lub połączenie przez tunel (`npx expo start --tunnel`). Zeskanowanie kodu QR z terminala otwiera natychmiast aplikację na urządzeniu z obsługą Live Reloading.

4.2. Budowanie Samodzielnego Pliku Instalacyjnego (.APK) Aby zainstalować aplikację na telefonie na stałe (bez konieczności połączenia z komputerem i bez Expo Go), wykorzystana zostanie darmowa usługa budowania w chmurze **EAS Build (Expo Application Services)**. Kroki do wygenerowania i instalacji APK:

1. Inicjalizacja konfiguracji EAS w katalogu projektu:

```
npx eas-cli build:configure
```

2. Konfiguracja pliku `eas.json` dla profilu testowego z wyjściem w postaci pliku APK:

```
{
  "build": {
```

```
    "preview": {  
      "android": {  
        "buildType": "apk"  
      }  
    }  
  }  
}
```

3. Uruchomienie budowania w chmurze:

```
npx eas-cli build -p android --profile preview
```

4. Po zakończeniu kompilacji w chmurze (ok. 5-10 minut) w terminalu pojawi się link oraz kod QR do pobrania gotowego pliku `.apk`.
5. Otwarcie linku na telefonie z Androidem, pobranie pliku i instalacja (wymaga jednorazowego zaznaczenia w Androidzie opcji: *Zezwalaj na instalację aplikacji z nieznanych źródeł*).